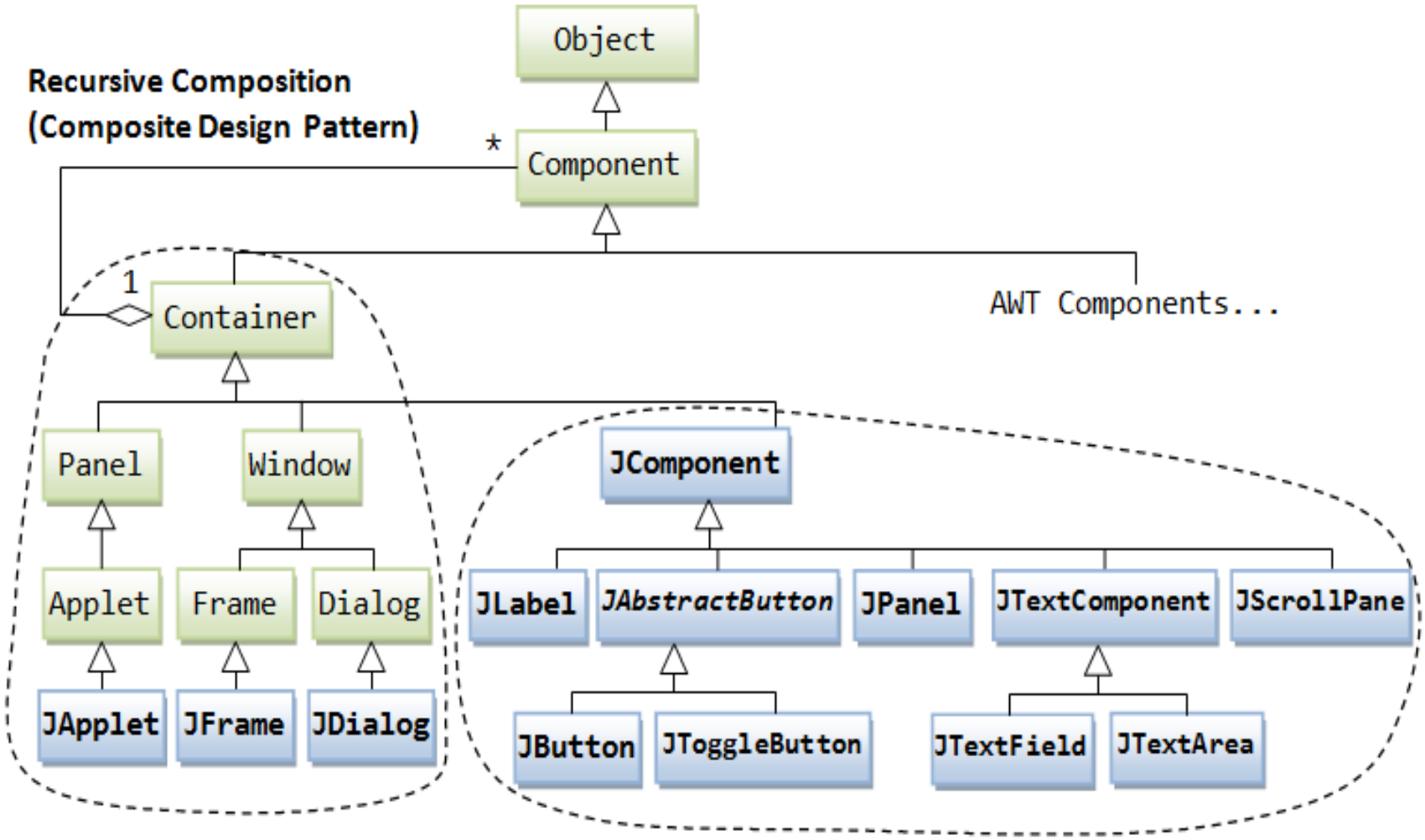
Java Swing

Introduction to Swing

- Package : javax.swing.*
- Swing is set of classes which provides more powerful and flexible components as compare to AWT.
- Build on top of AWT API and acts as replacement of AWT API.
- Swing component follows a **Model-View-Controller**
- Swing Components are implemented using Java and so they are platform independent.
- Called **lightweight** components

Introduction to Swing



Swing Features

- Swing introduces many innovations.
- **Borders** we can draw borders in many different styles around components using the setBorder( ) method.
- **Graphics Debugging** We can use setDebuggingGraphicsOptions method to set up graphics debugging which means, that you can watch each line as its drawn and make it flash.
- **Easy mouseless operation**: It is easy to connect keystrokes to components.

- **Tooltips** We can use the **setToolTipText** method of JComponent to give components a tooltip, one of those small windows that appear when the mouse hovers over a component and gives explanatory text.
- **Easy Scrolling** We can connect scrolling to various components-something that was impossible in AWT.
- **Pluggable look and feel** We can set the appearance of applets and applications to one of three standard looks. Windows, Motif (Unix) or Metal (Standard swing look).
- **New Layout Managers** Swing introduces the BoxLayout and OverlayLayout layout managers.

MVC Architecture

- Software design pattern for software development.
- **Model:**
  - Major function of this layer to maintain the data.
  - Database and logic.
- **View:**
  - Used to display full or partial data.
  - User Interface
- **Controller**:
  - Control the interaction and communication between Model and view.
  - Communication logic/integration logic

Difference Between AWT & Swing

- AWT uses **Applet and Frame** while Swing uses JApplet and JFrame for GUI.
- AWT is **platform dependent** code while Swing code is platform independent.
- Swing has bigger collection of classes and interfaces as compare to AWT.
- AWT components are **Heavyweight** where as Swing components are **Lightweight**.
- In Swing extra feature to Button: Provide Image.
- Swing provides: Tree, Table, Scrollpanes, Tabbedpanes, etc. new feature.
- AWT does not follow MVC.

JApplet

- Fundamental to Swing is the JApplet class, which extends Applet.
- Applets that use Swing must be subclasses of JApplet.
- JApplet is rich with functionality that is not found in Applet.
- For example, JApplet supports various “panes,” such as the **content pane**, the **glass pane**, and the **root pane**.

Methods

- When adding a component to an instance of JApplet, do not invoke the add( ) method of the applet instead, call add( ) for the content pane of the JApplet object.
- The content pane can be obtained via the method shown here:
  - **Container getContentPane( )**
- The add( ) method of Container can be used to add a component to a content pane.
  - **void add(comp)**
- Here, comp is the component to be added to the content pane.

Icons and Labels

- In Swing, icons are encapsulated by the ImageIcon class, which paints an icon from an image.
- Two of its constructors are shown here:
  - **ImageIcon(String filename)**
  - **ImageIcon(URL url)**
- The first form uses the image in the file named filename.
- The second form uses the image in the resource identified by url.

- The ImageIcon class implements the Icon interface that declares the methods shown here:
- int getIconHeight( )
  - Returns the height of the icon in pixels.
- int getIconWidth( )
  - Returns the width of the icon in pixels.
- void paintIcon(Component comp, Graphics g, int x, int y)
  - Paints the icon at position x,y on the graphics context g. Additional information about the paint operation can be provided in comp.

Swing labels

- Swing labels are instances of the JLabel class, which extends JComponent.
- It can display text and/or an icon.
- Some of its constructors are shown here:
  - **JLabel(Icon i)**
  - **JLabel(String s)**
  - **JLabel(String s, Icon i, int align)**
- Here, s and i are the text and icon used for the label.
- The align argument is either **LEFT, RIGHT,CENTER, LEADING, or TRAILING** these are constants.

- Icon getIcon( )
- String getText( )
- void setIcon(Icon i)
- void setText(String s)

```
import java.awt.*;
import javax.swing.*;
/* <applet code="JLabelDemo" width=250 height=150>
</applet> */
public class JLabelDemo extends JApplet
{
 public void init()
 {
 Container contentPane = getContentPane();
 ImageIcon ii = new ImageIcon("IC.jpg");
 JLabel jl = new JLabel("IC", ii, JLabel.CENTER);
 contentPane.add(jl);
 }
}
```

Text Fields

- The Swing text field is encapsulated by the JTextComponent class, which extends JComponent.
- It provides functionality that is common to Swing text components.
- One of its subclasses is JTextField, which allows us to edit one line of text.
- Some of its constructors are shown here:
  - JTextField( )
  - JTextField(int cols)
  - JTextField(String s, int cols)
  - JTextField(String s)

```
import java.awt.*;
import javax.swing.*;
/* <applet code="JTextFieldDemo" width=300 height=50>
</applet>
*/
public class JTextFieldDemo extends Japplet {
 JTextField jtf;
 public void init()  {
 Container contentPane = getContentPane();
 contentPane.setLayout(new FlowLayout());
 jtf = new JTextField(15);
 contentPane.add(jtf);
 } }
```

Button

- Swing buttons provide features that are not found in the Button class defined by the AWT. For example, we can associate an icon with a Swing button.
- Swing buttons are subclasses of the AbstractButton class, which extends JComponent. AbstractButton contains many methods that allow us to control the behavior of buttons, check box and radio buttons.
- For example, we can define different icons that are displayed for the component when it is disabled, pressed, or selected.
- Another icon can be used as rollover icon, which is displayed when the mouse is positioned over that component.

Methods

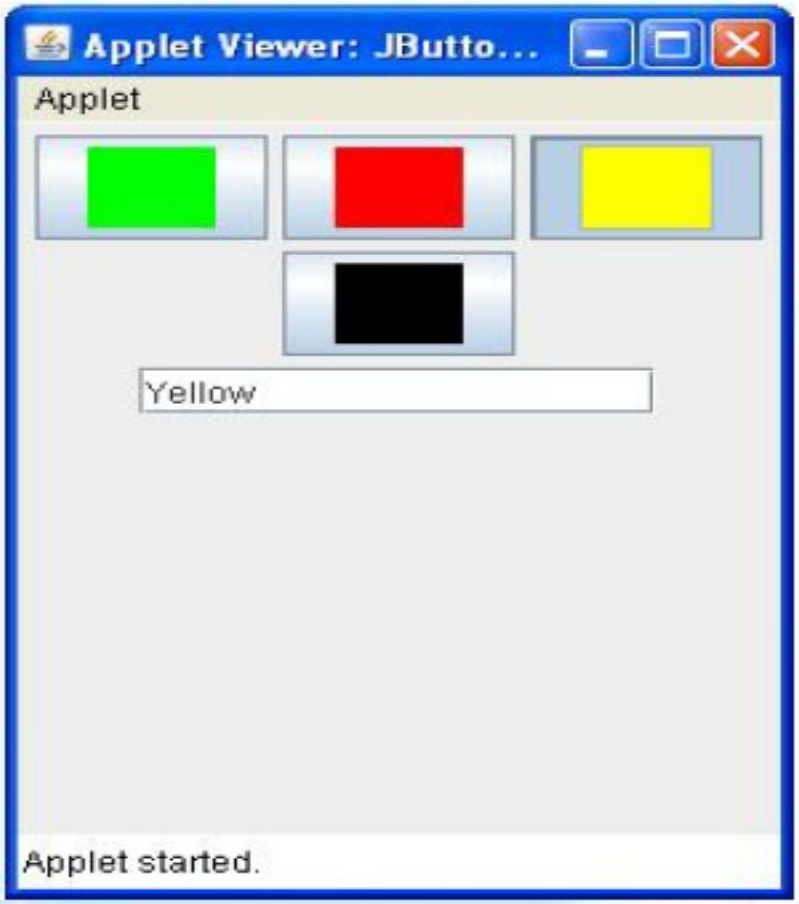
- void setDisabledIcon(Icon di)
- void setPressedIcon(Icon pi)
- void setSelectedIcon(Icon si)
- void setRolloverIcon(Icon ri)
- String getText( )
- void setText(String s)
- Concrete subclasses of AbstractButton generate action events when they are pressed. Listeners register and un-register for these events via the methods shown here:
- void addActionListener(ActionListener al)
- void removeActionListener(ActionListener al)
- Here, *al* is the action listener.

JButton

- The JButton class provides the functionality of a push button.
- JButton allows an icon, string, or both to be associated with the push button.
- Some of its constructors are shown here:
- JButton(Icon i)
- JButton(String s)
- JButton(String s, Icon i)

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
/* <applet code="JButtonDemo" width=250 height=300> </applet>
*/
public class JButtonDemo extends JApplet
implements ActionListener {
JTextField jtf;
public void init() {
Container contentPane = getContentPane();
contentPane.setLayout(new FlowLayout());
ImageIcon france = new ImageIcon("green.jpg");
JButton jb = new JButton(france);
jb.setActionCommand("Green");
jb.addActionListener(this);
contentPane.add(jb);
```

```
ImageIcon germany = new
ImageIcon("red.jpg");
jb = new JButton(germany);
jb.setActionCommand("Red");
jb.addActionListener(this);
contentPane.add(jb);
ImageIcon italy = new
ImageIcon("yellow.jpg");
jb = new JButton(italy);
jb.setActionCommand("Yellow");
jb.addActionListener(this);
contentPane.add(jb);
ImageIcon japan = new
ImageIcon("black.jpg");
jb = new JButton(japan);
jb.setActionCommand("Black");
jb.addActionListener(this);
contentPane.add(jb);
jtf = new JTextField(15);
contentPane.add(jtf);
}
public void
actionPerformed(ActionEvent ae)
{
jtf.setText(ae.getActionCommand()
); }
}
```


Applet Viewer: JButto...
Applet
Yellow
Applet started.

JCheckBox

- The **JCheckBox** class, which provides the functionality of a check box, is a concrete implementation of AbstractButton.
- **It is immediate super-class is JToggleButton, which provides support for two-state buttons**.
- Some of its constructors are shown here:
- JCheckBox(Icon i)
- JCheckBox(Icon i, boolean state)
- JCheckBox(String s)
- JCheckBox(String s, boolean state)
- JCheckBox(String s, Icon i)
- JCheckBox(String s, Icon i, boolean state)

- The state of the check box can be changed via the following method:
- void setSelected(boolean state)
- When a check box is selected or deselected, an item event is generated. This is handled by itemStateChanged( ).
- Inside itemStateChanged( ), the getItem( ) method gets the JCheckBox object that generated the event.
- The getText( ) method gets the text for that check box.

```
public class JCheckBoxDemo extends JApplet
implements ItemListener {
JTextField jtf;
public void init() {
Container contentPane = getContentPane();
contentPane.setLayout(new FlowLayout());
JCheckBox cb = new JCheckBox("C", true);
cb.addItemListener(this);
contentPane.add(cb);
cb = new JCheckBox("C++", false);
cb.addItemListener(this);
contentPane.add(cb);
cb = new JCheckBox("Java", false);
cb.addItemListener(this);
contentPane.add(cb);
cb = new JCheckBox("Perl", false);
cb.addItemListener(this);
contentPane.add(cb);
jtf = new JTextField(15);
contentPane.add(jtf);
}
```

```
public void itemStateChanged(ItemEvent ie)
{
JCheckBox cb = (JCheckBox)ie.getItem();
jtf.setText(cb.getText());
}
}
```

Radio Buttons

- Radio buttons are supported by the JRadioButton class, which is a concrete implementation of AbstractButton. Its immediate super-class is JToggleButton, which provides support for two-state buttons.
- Some of its constructors are shown here:
- JRadioButton(Icon i)
- JRadioButton(Icon i, boolean state)
- JRadioButton(String s)
- JRadioButton(String s, boolean state)
- JRadioButton(String s, Icon i)
- JRadioButton(String s, Icon i, boolean state)

- Radio buttons must be configured into a group. Only one of the buttons in that group can be selected at any time.
- The ButtonGroup class is instantiated to create a button group. Its default constructor is invoked for this purpose.
- Elements are then added to the button group via the following method:
- void add(AbstractButton ab)

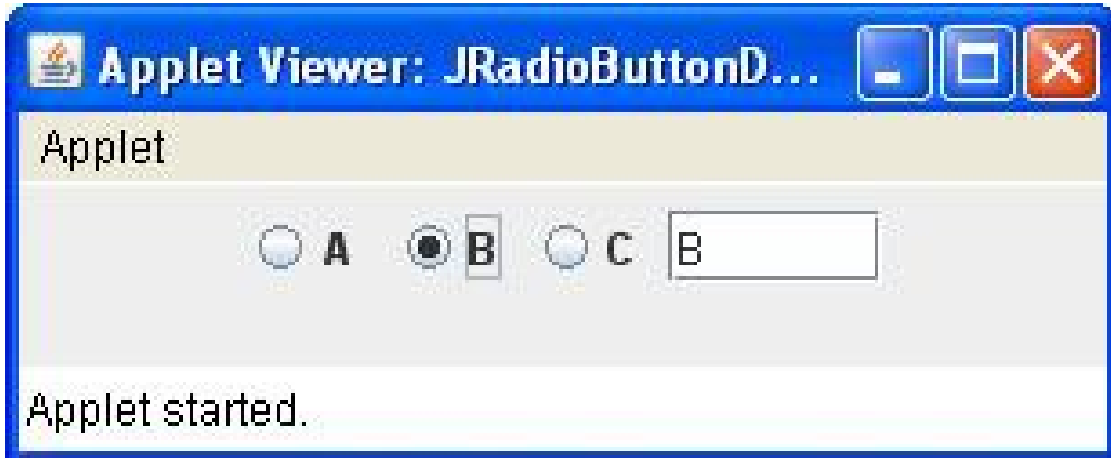
```
public class JRadioButtonDemo extends JApplet
implements ActionListener {
JTextField tf;
public void init() {
Container contentPane = getContentPane();
contentPane.setLayout(new FlowLayout());
JRadioButton b1 = new JRadioButton("A");
b1.addActionListener(this);
contentPane.add(b1);

JRadioButton b2 = new JRadioButton("B");
b2.addActionListener(this);
contentPane.add(b2);

JRadioButton b3 = new JRadioButton("C");
b3.addActionListener(this);
contentPane.add(b3);
```

```
ButtonGroup bg = new
ButtonGroup();
bg.add(b1);
bg.add(b2);
bg.add(b3);
tf = new JTextField(5);
contentPane.add(tf);
}
```

```
public void
actionPerformed(ActionEvent ae)
{
tf.setText(ae.getActionCommand(
));
}
}
```


Applet Viewer: JRadioButtonD...
Applet
A
B
C
B
Applet started.

JComboBox

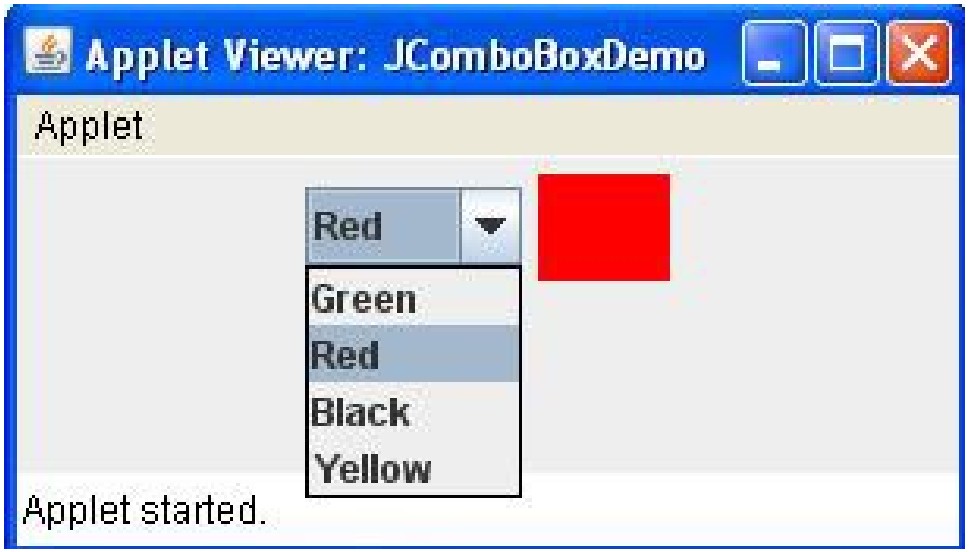
- Swing provides a combo box (a combination of a text field and a dropdown list) through the JComboBox class, which extends JComponent.
- A combo box normally displays one entry. However, it can also display a drop-down list that allows a user to select a different entry. We can also type our selection into the text field.
- Two of JComboBox's constructors are shown here:
- JComboBox( )
- JComboBox(Vector v)
- JComboBox(Objects obj[])
- Here, *v* is a vector that initializes the combo box and *obj* is the array of objects.

- Items are added to the list of choices via the addItem( ) method:

- void addItem(Object obj)
- public void setEditable(boolean aFlag)
  - It determines whether the JComboBox field is editable or not?
- public boolean isEditable()
  - It returns true if the JComboBox is editable. By default, a combo box is not editable.
- public void setMaximumRowCount(int count)
  - It sets the maximum nmber of rows the JComboBox displays.
- Public void setSelectedItem(Object anObject)
- public void insertItemAt(Object anObject, int index)
- public void removeItem(Object anObject)
- public void removeItemAt(int anIndex)

```
public class JComboBoxDemo extends JApplet
implements ItemListener {
JLabel jl;
ImageIcon green, red, black, yellow;
public void init() {
Container contentPane = getContentPane();
contentPane.setLayout(new FlowLayout());
JComboBox jc = new JComboBox();
jc.addItem("Green");
jc.addItem("Red");
jc.addItem("Black");
jc.addItem("Yellow");
jc.addItemListener(this);
contentPane.add(jc);
jl = new JLabel(new ImageIcon("green.jpg"));
contentPane.add(jl);
}
```

```
public void
itemStateChanged(ItemEvent ie)
{
String s = (String)ie.getItem();
jl.setIcon(new ImageIcon(s + ".jpg"));
}
}
```


Applet Viewer: JComboBoxDemo
Applet
Red
Green
Red
Black
Yellow
Applet started.

Advanced Swing Components

Advanced Swing Components

Tabbed Panes

- A tabbed pane is a component that appears as a **group of folders in a file cabinet.**
  - Each folder has a title.
  - When a user selects a folder, its contents become visible.
  - Only one of the folders may be selected at a time.
  - Tabbed panes are commonly used for setting configuration options.
- Tabbed panes are encapsulated by the JTabbedPane class, which extends JComponent.

JTabbedPane

- There are three constructors of JTabbedPane.
  - **JTabbedPane()**
  - **JTabbedPane(int tabPlacement)**
- The first form creates an empty TabbedPane with a default tab placement of JTabbedPane -TOP.
- Second form creates an empty TabbedPane with the specified tab placement :
  - JTabbedPane.TOP
  - JTabbedPane.BOTTOM
  - JTabbedPane.LEFT
  - JTabbedPane.RIGHT

JTabbedPane

- **JTabbedPane(int tabPlacement, int tabLayoutPolicy)**

- Tab layout policy may be either of the following:
- JTabbedPane.WRAP_TAB_LAYOUT
- JTabbedPane.SCROLL_TAB_LAYOUT

- The general procedure to use a tabbed pane in an applet is outlined here:

1. Create a JTabbedPane object.
2. Call addTab( ) to add a tab to the pane. (The arguments to this method define the title of the tab and the component it contains.)
3. Repeat step 2 for each tab.
4. Add the tabbed pane to the content pane of the applet.

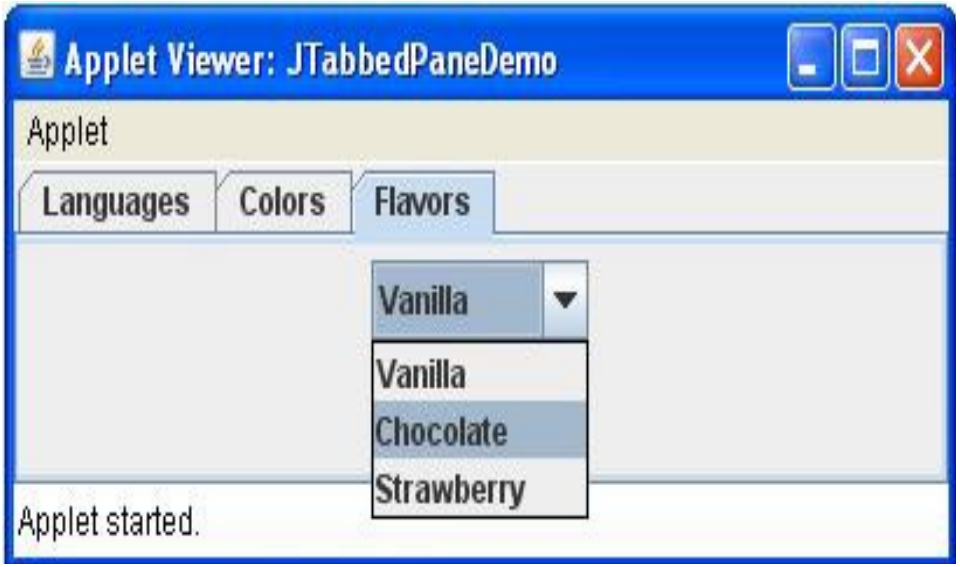
- Tabs are defined via the following method:
- **void addTab(String str, Component comp)**
  - Here, *str* is the title for the tab, and
  - comp is the component that should be added to the tab.
- Typically, a JPanel or a subclass of it is added.

```
import javax.swing.*;
/*<applet code="JTabbedPaneDemo" width=400 height=100> </applet> */
public class JTabbedPaneDemo extends Japplet {
public void init() {
JTabbedPane jtp = new JTabbedPane();
jtp.addTab("Languages", new LangPanel());
jtp.addTab("Colors", new ColorsPanel());
jtp.addTab("Flavors", new FlavorsPanel());
getContentPane().add(jtp);
}
}
```

```
class LangPanel extends JPanel
{
public LangPanel()
{
JButton b1 = new JButton("Marathi");
add(b1);
JButton b2 = new JButton("Hindi");
add(b2);
JButton b3 = new JButton("Bengali");
add(b3);
JButton b4 = new JButton("Tamil");
add(b4);
}
}
```

```
class ColorsPanel extends JPanel
{
public ColorsPanel()
{
JCheckBox cb1 = new JCheckBox("Red");
add(cb1);
JCheckBox cb2 = new JCheckBox("Green");
add(cb2);
JCheckBox cb3 = new JCheckBox("Blue");
add(cb3);
}
}
```

```
class FlavorsPanel extends JPanel
{
public FlavorsPanel()
{
JComboBox jcb = new JComboBox();
jcb.addItem("Vanilla");
jcb.addItem("Chocolate");
jcb.addItem("Strawberry");
add(jcb);
}
}
```



Scroll Panes

- A scroll pane is a component that presents a rectangular area in which a component may be viewed.
- Horizontal and/or vertical scroll bars may be provided if necessary.
- Scroll panes are implemented in Swing by the **JScrollPane** class, which extends JComponent.

Constructors

- **JScrollPane()**
- **JScrollPane(Component comp)**
- **JScrollPane(int vsb, int hsb)**
- **JScrollPane(Component comp, int vsb, int hsb)**
- Here, *comp* is component to add to the scroll pane.
- ***vsb* and *hsb*** are int constants that define when vertical and horizontal scroll bars for this scroll pane are shown.
- These constants are defined by the **ScrollPaneConstants** interface.
  - HORIZONTAL_SCROLLBAR_ALWAYS
  - HORIZONTAL_SCROLLBAR_AS_NEEDED
  - VERTICAL_SCROLLBAR_ALWAYS
  - VERTICAL_SCROLLBAR_AS_NEEDED

- Here are the steps that you should follow to use a scroll pane in an applet:

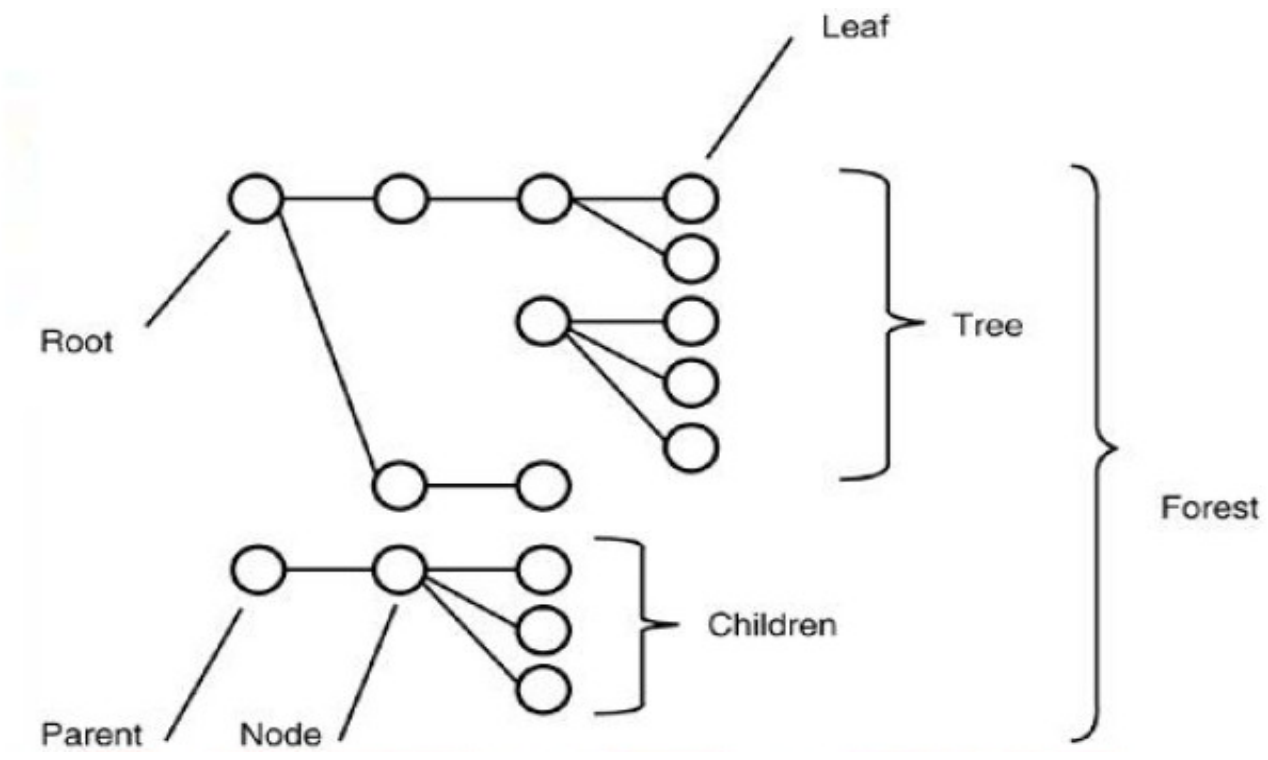
1. Create a JComponent object.
2. Create a JScrollPane object. (The arguments to the constructor specify the component and the policies for vertical and horizontal scroll bars.)
3. Add the scroll pane to the content pane of the applet.

```
public class JScrollPaneDemo extends Japplet {
public void init(){
Container contentPane = getContentPane();
contentPane.setLayout(new BorderLayout());
JPanel jp = new JPanel();
jp.setLayout(new GridLayout(20, 20));
int b = 0;
for(int i = 0; i < 20; i++) {
for(int j = 0; j < 20; j++) {
jp.add(new JButton("Button " + b));
++b; } }
int v = ScrollPaneConstants.VERTICAL_SCROLLBAR_AS_NEEDED;
int h = ScrollPaneConstants.HORIZONTAL_SCROLLBAR_AS_NEEDED;
JScrollPane jsp = new JScrollPane(jp, v, h);
contentPane.add(jsp, BorderLayout.CENTER);
} }
```

Applet Viewer: JScrollPaneDemo
Applet
tton 64
Button 65
Button 66
tton 84
Button 85
Button 86
ton 104
Button 105
Button 106
ton 124
Button 125
Button 126
ton 144
Button 145
Button 146
ton 164
Button 165
Button 166
ton 184
Button 185
Button 186
ton 204
Button 205
Button 206
Applet started.

Trees

- A tree is a component that presents a hierarchical view of data.
- A user has the ability to expand or collapse individual sub-trees in this display.



JTree

- Trees are implemented in Swing by JTree class, which extends

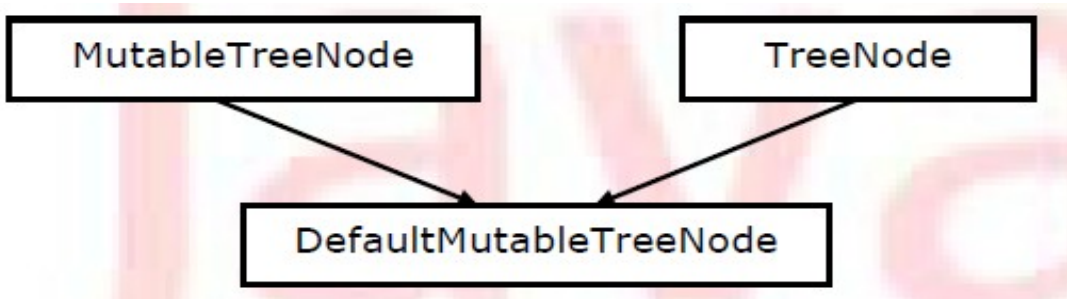
JComponent.

  - **JTree(Hashtable ht)**
  - **JTree(Object obj[ ])**
  - **JTree(TreeNode tn)**
  - **JTree(Vector v)**

- The first form creates a tree in which each element of the hash table *ht* is a child node.
- Each element of array *obj* is a child node in the second form.
- The tree node ***tn* is the root** of the tree in the third form.
- Finally, the last form uses the elements of vector *v* as child nodes.

- A JTree object generates events when a node is expanded or collapsed. The following methods allow listeners to register and unregister for these notifications.
- **void addTreeExpansionListener(TreeExpansionListener tel)**
- **void removeTreeExpansionListener(TreeExpansionListener tel)**
  - Here, *tel* is the listener object.
- The getPathForLocation( ) method is used to translate a mouse click on a specific point of the tree to a tree path.
- **TreePath getPathForLocation(int x, int y)**
- Here, *x* and *y* are the coordinates at which the mouse is clicked.
- The return value is a TreePath object that encapsulates information about the tree node that was selected by the user.

- The TreePath class encapsulates information about a path to a particular node in a tree. It provides several constructors and methods.
- The TreeNode interface declares methods that obtain information about a tree node.
- For example, it is possible to obtain a reference to the parent node or an enumeration of the child nodes.
- The MutableTreeNode interface extends TreeNode.
- It declares methods that can insert and remove child nodes or change the parent node.

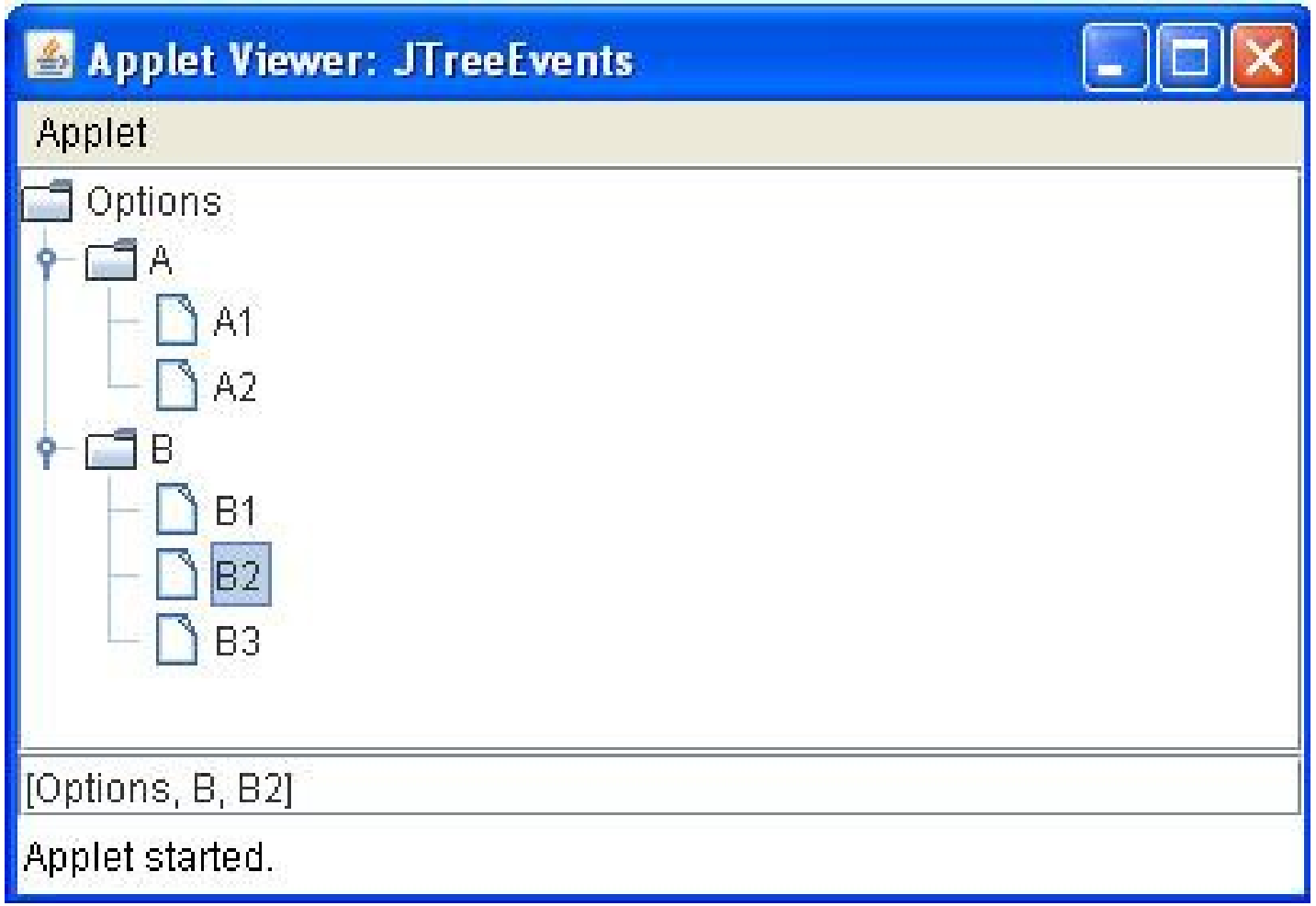


- The DefaultMutableTreeNode class implements the MutableTreeNode interface.
- It represents a node in a tree.
- **DefaultMutableTreeNode(Object obj)**
  - Here, *obj* is the object to be enclosed in this tree node.
- The new tree node doesn't have a parent or children.
- To create a hierarchy of tree nodes, the add( ) method of DefaultMutableTreeNode can be used.
- **void add(MutableTreeNode child)**

- Tree expansion events are described by the class TreeExpansionEvent in
- the javax.swing.event package.
- The getPath( ) method of this class returns a TreePath object that describes the path to the changed node.
- **TreePath getPath( )**
- The TreeExpansionListener interface provides the following two methods:
- **void treeCollapsed(TreeExpansionEvent tee)**
- **void treeExpanded(TreeExpansionEvent tee)**
- Here, *tee* is the tree expansion event.
- The first method is called when a sub-tree is hidden, and the second method is called when a sub-tree becomes visible.

- Here are the steps that we should follow to use a tree in an applet:

1. Create a JTree object.
2. Create a JScrollPane object. (The arguments to the constructor specify the tree and the policies for vertical and horizontal scroll bars.)
3. Add the tree to the scroll pane.
4. Add the scroll pane to the content pane of the applet.


Applet Viewer: JTreeEvents
Applet
Options
A
A1
A2
B
B1
B2
B3
[Options, B, B2]
Applet started.

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.tree.*;
/* <applet code="JTreeEvents" width=400 height=200>
</applet>
*/
public class JTreeEvents extends JApplet
{
JTree tree;
JTextField jtf;
public void init()
{
```

```
Container contentPane=getContentPane();
contentPane.setLayout(new BorderLayout());
DefaultMutableTreeNode top=new DefaultMutableTreeNode("Options");
DefaultMutableTreeNode a= new DefaultMutableTreeNode("A");
top.add(a);
DefaultMutableTreeNode a1=new DefaultMutableTreeNode("A1");
a.add(a1);
DefaultMutableTreeNode a2=new DefaultMutableTreeNode("A2");
a.add(a2);
DefaultMutableTreeNode b= new DefaultMutableTreeNode("B");
top.add(b);
DefaultMutableTreeNode b1=new DefaultMutableTreeNode("B1");
b.add(b1);
DefaultMutableTreeNode b2=new DefaultMutableTreeNode("B2");
b.add(b2);
DefaultMutableTreeNode b3=new DefaultMutableTreeNode("B3");
b.add(b3);
```

```
tree=new JTree(top);
int v=ScrollPaneConstants.VERTICAL_SCROLLBAR_AS_NEEDED;
int h=ScrollPaneConstants.HORIZONTAL_SCROLLBAR_AS_NEEDED;
JScrollPane jsp=new JScrollPane(tree,v,h);
contentPane.add(jsp,BorderLayout.CENTER);
jtf=new JTextField("",20);
contentPane.add(jtf,BorderLayout.SOUTH);

tree.addMouseListener(new MouseAdapter() {
public void mouseClicked(MouseEvent me) {
doMouseClicked(me);
}
});
}
```

```
void doMouseClicked(MouseEvent me)
{
TreePath p=tree.getPathForLocation(me.getX(),me.getY());
if(tp!=null)
jtf.setText(tp.toString());
else
jtf.setText("");
}
}
```

Tables

- A table is a component that displays rows and columns of data. We can
- drag the cursor on column boundaries to resize columns. We can also drag a
- column to a new position. Tables are implemented by the JTable class, which
- extends JComponent. One of its constructors is shown here:
- JTable(Object data[ ][ ], Object colHeads[ ])
- JTable(int numRows, int numColumns)
- JTable(Vector rowData, Vector columnData)

- the steps for using a table in an applet:

1) Create a JTable object.

2) Create a JScrollPane object. (The arguments to the constructor specify

the table and the policies for vertical and horizontal scroll bars.)

3) Add the table to the scroll pane.

4) Add the scroll pane to the content pane of the applet.

```
import java.awt.*;
import javax.swing.*;
/* <applet code="JTableDemo" width=400 height=200>
</applet> */
public class JTableDemo extends JApplet
{
public void init()
{
Container contentPane = getContentPane();
contentPane.setLayout(new BorderLayout());
final String[] colHeads = { "Name", "Phone", "Fax" };
final Object[][] data = {
```

```
{ "Pramod", "4567", "8675" },
{ "Tausif", "7566", "5555" },
{ "Nitin", "5634", "5887" },
{ "Amol", "7345", "9222" },
{ "Vijai", "1237", "3333" },
{ "Ranie", "5656", "3144" },
{ "Mangesh", "5672", "2176" },
};
JTable table = new JTable(data, colHeads);
int v = ScrollPaneConstants.VERTICAL_SCROLLBAR_AS_NEEDED;
int h = ScrollPaneConstants.HORIZONTAL_SCROLLBAR_AS_NEEDED;
JScrollPane jsp = new JScrollPane(table, v, h);
contentPane.add(jsp, BorderLayout.CENTER);
}
}
```

Applet Viewer: JTableDemo

Applet

Name	Phone	Fax
Tausif	7566	5555
Nitin	5634	5887
Amol	7345	9222
Vijai	1237	3333
Ranie	5656	3144
Mangesh	5672	2176
Suhail	6741	4244
Nilofer	9023	5159
Jinnie	1134	5332
Heena	5689	1212
Saurav	9030	1313

Applet started.

JProgressBar

- JProgressBar visually displays the progress of some specified task.
- JProgressBar shows the percentage of completion of specified task.
- The progress bar fills up as the task reaches it completion.
- In addition to show the percentage of completion of task, it can also display some text .
- **Constructors of JProgressBar :**
- **JProgressBar()** : creates an progress bar with no text on it;
- **JProgressBar(int orientation)** orientation - SwingConstants.VERTICAL, SwingConstants.HORIZONTAL
- **JProgressBar(int min, int max)**
- **JProgressBar(int orientation, int min, int max)**

- **methods of JProgressBar are :**
- **int getMaximum() :** returns the progress bar's maximum value.
- **int getMinimum()** : returns the progress bar's minimum value.
- **String getString()** : get the progress bar's string representation of current value.
- **void setMaximum(int n)** : sets the progress bar's maximum value to the value n.
- **void setMinimum(int n)** : sets the progress bar's minimum value to the value n.
- **void setValue(int n)** : set Progress bar's current value to the value n.
- **void setString(String s)** : set the value of the progress String to the String s.

```
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;
public class progress extends JFrame {
   static JFrame f;
   static JProgressBar b;
public static void main()
   {
        f = new JFrame("ProgressBar demo");
        JPanel p = new JPanel();
        b = new JProgressBar();
        b.setValue(0);
```

```
b.setStringPainted(true);
        p.add(b);
       f.add(p);
        f.setSize(500, 500);
      f.setVisible(true);
        fill();
   }
     public static void fill()     {
      int i = 0;
      try {
         while (i <= 100) {
                    b.setValue(i + 10);
              Thread.sleep(1000);
            i += 20;
         }        }         catch (Exception e) {
      }    }  }
```